



Hitachi Portal Integration (iOS)

For Internal Use Only

Hitachi Portal Integration (iOS)

Confidentiality

This document contains proprietary information that is confidential to Hitachi Asia. Disclosure of this document in full or in part, may result in material damage to Hitachi Asia. Written permission must be obtained from Hitachi Asia prior to the disclosure of this document to a third party.

Version: 0.5 **File Name:** Hitachi Portal Integration (iOS)_v0.5 **Confidentiality Category:** For Internal Use Only

Document Change Control

Version	Date	Author(s)	Description
0.1	15 July 2024	Chris Lee	Initial Version
0.2	30 July 2024	Chris Lee	Update Minimum Version and addEventListener
0.3	6 Aug 2024	Chris Lee	Update Notify Listener
0.4	17 Oct 2025	Chris Lee	Add Custom Header/Title, Permissions (Contacts, Photo Library), and Stacked Portals
0.5	5 Mar 2026	Chris Lee	Add Geolocation support

Table of Contents

[Introduction](#)

[1.1 Overview](#)

[1.2 What's New in v0.5](#)

[Specification](#)

[Installation](#)

[3.1 Add the Framework to Your Project](#)

[3.2 Update Build Settings](#)

[3.3 Configure Info.plist Permissions](#)

[Example Usage](#)

[4.1 Basic Portal](#)

[4.2 Portal with Custom Header and Title](#)

[4.3 Handling Permissions \(Contacts\)](#)

[4.4 Handling Photo Library Access](#)

[4.5 Geolocation](#)

[API Reference](#)

[5.1 Portal](#)

[5.2 open](#)

[5.3 close](#)

[5.4 addEventListener](#)

[5.5 notifyListener](#)

1. Introduction

1.1 Overview

This document provides a comprehensive overview of the HASPortal SDK, highlighting its primary features and benefits. The SDK includes functionalities such as Wrapping, Token Interceptor, and Publish-Subscribe to enhance the overall performance, security, and scalability of the application.

Objectives: - **Encapsulation and Security:** The SDK ensures that the portal functionality is encapsulated within a secure environment, providing better control over lifecycle events and consistent integration across different parts of the application. - **Enhanced Security and Streamlined**

Authentication: The Token Interceptor automates the management of authentication tokens, simplifying user session handling and ensuring secure communication between the app and backend services. -

Asynchronous Communication and Decoupling: The Pub-Sub mechanism facilitates non-blocking, event-driven communication, promoting a modular and scalable architecture by decoupling event producers and consumers.

Key Benefits: - **Encapsulation:** Keeps portal interactions secure and consistent, with enhanced control over initialization, usage, and termination. - **Security and Efficiency:** Token management, reducing the risk of errors and ensuring secure network communications. - **Modularity and Flexibility:** Supports dynamic addition and removal of event listeners, improving application responsiveness and maintainability through asynchronous event handling.

1.2 What's New in v0.5

1. **Geolocation Support (iOS 15+)**
 2. Web content can now use `navigator.geolocation` to access device location
 3. The SDK automatically handles the WKWebView geolocation permission delegate
 4. Host app only needs to add `NSLocationWhenInUseUsageDescription` to Info.plist
 5. No native event handling code required — the standard browser Geolocation API works directly
 6. Permission prompt shown automatically when web content requests location
 7. If permission was previously denied, the SDK shows a Settings redirect alert
-

2. Specification

- **iOS minimum operating system version:** 12.0
 - **SDK / xcframework Size:** 1.7mb
 - **Photo Library Support:** iOS 14.0+
 - **Geolocation Support:** iOS 15.0+
-

3. Installation

3.1 Add the Framework to Your Project

1. Open your project in Xcode.
2. In the Project Navigator, right-click on your project name and select **Add Files to "YourProjectName"**.
3. Navigate to the location of your `HASPortal.xcframework` file and select it.
4. Ensure that the **Copy items if needed** option is checked, and add it to your target(s).

3.2 Update Build Settings

1. Select your project in the Project Navigator, then select your app target.
2. Go to the **Build Phases** tab.
3. In the **Link Binary With Libraries** section, ensure your xcframework is listed. If not, click the **+** button, find your xcframework, and add it.

3.3 Configure Info.plist Permissions

To use the permission features, you must add the following keys to your `Info.plist` :

For Contacts Access:

```
<key>NSContactsUsageDescription</key>
<string>We need access to your contacts to [explain your use case]</string>
```

For Photo Library Access:

```
<key>NSPhotoLibraryUsageDescription</key>
<string>We need access to your photo library to [explain your use case]</string>
```

For Camera Access:

```
<key>NSCameraUsageDescription</key>
<string>We need camera access to capture photos and videos</string>
```

For Microphone Access (optional):

```
<key>NSMicrophoneUsageDescription</key>  
<string>We need microphone access to record audio</string>
```

For Geolocation Access:

```
<key>NSLocationWhenInUseUsageDescription</key>  
<string>We need location access to [explain your use case]</string>
```

Note: The SDK will not request permissions if these keys are not present in your Info.plist. Camera, microphone, and geolocation permissions are handled automatically by WKWebView when web content requests access (iOS 15+).

4. Example Usage

4.1 Basic Portal

Below is a basic example of how to use the HASPortal SDK:

```
import UIKit
import HASPortal

class ViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()

        let configuration: [String: Any] = [
            "accessToken": "your-access-token-here"
        ]
        let portal = Portal(configuration: configuration)

        portal.addEventListener(eventName: "eventsCallbacks") { data in
            print("Listener received event data: \(data)")

            if let eventData = data as? [String: Any],
                let name = eventData["name"] as? String {

                if name == "close" {
                    print("Handling close event")
                    portal.close()
                } else if name == "token-expired" {
                    print("Handling Invalid Token event")
                    let newConfiguration: [String: Any] = [
                        "accessToken": "new-access-token"
                    ]
                    portal.notifyListener(eventName: "token-updated", data: newConfiguration)
                }
            }
            return nil
        }

        let urlString = "https://example.com"
        portal.open(urlString: urlString)
    }
}
```

4.2 Portal with Custom Header and Title

The SDK now supports displaying a custom native header with a title:

```
import UIKit
import HASPortal

class ViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()

        let configuration: [String: Any] = [
            "accessToken": "your-access-token-here",
            "title": "My Portal", // Title shown in header
            "showHeader": true // Enable custom header
        ]
        let portal = Portal(configuration: configuration)
    }
}
```

```
portal.addEventListener(eventName: "eventsCallbacks") { data in
    // Handle events...
    return nil
}

// You can also specify header options in the open() method
portal.open(
    urlString: "https://example.com",
    showHeader: true, // Show custom header
    title: "My Portal", // Header title
    backgroundColor: "#E20079" // Custom background color (hex)
)
}
```

Header Features: - Displays a native iOS header at the top of the portal - Automatically adjusts text color (black/white) based on background brightness - Includes a back button (chevron icon) for navigation - 52pt content height + safe area inset (works with notched devices) - Rounded bottom corners for modern appearance

4.3 Handling Permissions (Contacts)

The SDK supports requesting device contacts permission and fetching contact data. The SDK handles the permission flow automatically — no additional native code is needed beyond the standard event listener.

```
portal.addEventListener(eventName: "eventsCallbacks") { data in
    if let eventData = data as? [String: Any],
        let name = eventData["name"] as? String {

        // The SDK automatically handles the "permission" event
        // No additional native code needed!
        // Users will see:
        // 1. System permission prompt (if not determined)
        // 2. Settings alert (if previously denied)
        // 3. Contacts data sent back to web (if authorized)
    }
    return nil
}
```

Permission Flow: 1. Web dispatches `permission` event with `type: "contact"` 2. SDK checks authorization status 3. If not determined: Shows system permission prompt 4. If denied/restricted: Shows alert with Settings button 5. If authorized: Fetches contacts and sends to web via `cap-events-response`

4.4 Handling Photo Library Access

The SDK supports selecting photos from the device photo library (iOS 14+). When web content requests photo library access, the SDK automatically presents the native `PHPickerViewController`. No additional native code is needed beyond the standard event listener and the `NSPhotoLibraryUsageDescription` key in Info.plist.

Note: Photo library picker requires iOS 14.0 or later. On older iOS versions, the SDK will return an error response.

4.5 Geolocation

The SDK supports geolocation in web content loaded within the portal (iOS 15+). The SDK handles the WKWebView geolocation delegate automatically — no native event code is required from the host app.

Host App Setup

Add the location usage description to your `Info.plist`:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>This app needs location access to show your position on the map.</string>
```

This is the **only** native-side requirement. No additional Swift code is needed.

Permission Flow: 1. Web content requests geolocation access 2. WKWebView triggers the SDK's geolocation permission delegate 3. If not determined: Shows the iOS location permission prompt 4. If denied/restricted: Shows an alert with a Settings redirect button 5. If authorized: Geolocation data flows directly to the web content

Important Notes: - Requires iOS 15.0 or later - The `NSLocationWhenInUseUsageDescription` key **must** be present in Info.plist, otherwise iOS will silently deny the permission request - Geolocation does **not** use the custom event system (`cap-events`). It is handled automatically by the SDK's WKWebView delegate

5. API Reference

5.1 Portal

Constructor

```
class Portal(configuration: [String: Any]) -> Portal
```

Creates a new Portal instance with the specified configuration.

Parameters

Parameter	Type	Description
configuration	[String: Any]	Configuration dictionary containing authentication and UI options

Configuration Keys

Key	Type	Required	Description
accessToken	String	Yes*	Authentication token (set as HttpOnly cookie)
betaUser	String	No	Beta user identifier (alternative to accessToken)
title	String	No	Title to display in custom header

Key	Type	Required	Description
showHeader	Bool	No	Whether to show custom header (default: false)
theme	String	No	Theme name (injected to sessionStorage)
event	[String: Any]	No	Event data (injected to sessionStorage)

*Either `accessToken` or `betaUser` must be provided.

Sample

```
var configuration: [String: Any] = [
    "accessToken": "your-access-token",
    "title": "My Portal",
    "showHeader": true
]
let portal = Portal(configuration: configuration)
```

5.2 open

Opens the portal with the specified URL and options.

Signature

```
func open(
    urlString: String,
    headers: [String: String] = [:],
    closeModal: Bool = false,
    closeModalTitle: String = "",
    closeModalDescription: String = "",
    closeModalOk: String = "",
    closeModalCancel: String = "",
    isInspectable: Bool = false,
    isAnimated: Bool = true,
    shareDisclaimer: [String: Any]? = nil,
    toolbarType: String = "blank",
    backgroundColor: String = "#FFFFFF",
    ignoreUntrustedSSLError: Bool = false,
    showReloadButton: Bool = false,
    showHeader: Bool = false,
    title: String = ""
) -> Void
```

Parameters

Parameter	Type	Default	Description
urlString	String	(required)	URL to open in the portal
headers	[String: String]	[:]	Custom HTTP headers
closeModal	Bool	false	Show confirmation dialog on close
closeModalTitle	String	""	Close confirmation title
closeModalDescription	String	""	Close confirmation message

Parameter	Type	Default	Description
closeModalOk	String	""	Close confirmation OK button text
closeModalCancel	String	""	Close confirmation Cancel button text
isInspectable	Bool	false	Enable Web Inspector (debug builds)
isAnimated	Bool	true	Animate portal presentation
shareDisclaimer	[String: Any]?	nil	Share button disclaimer configuration
toolbarType	String	"blank"	Toolbar type: "blank", "navigation", "activity", "progress"
backgroundColor	String	"#FFFFFF"	Header background color (hex format)
ignoreUntrustedSSLError	Bool	false	Bypass SSL certificate validation (use cautiously)
showReloadButton	Bool	false	Show reload button in toolbar
showHeader	Bool	false	Show custom native header
title	String	""	Title text for custom header

Sample

```
portal.open(
  urlString: "https://example.com",
  showHeader: true,
  title: "My Portal",
  backgroundColor: "#E20079",
  toolbarType: "blank"
)
```

Toolbar Types: - "blank" - No toolbar (full screen content) - "navigation" - Back/forward navigation buttons - "activity" - Share button - "progress" - Progress bar

Note: Parameter values override configuration values. If both are provided, the parameter takes precedence.

5.3 close

Closes the currently open portal.

Signature

```
func close() -> Void
```

Sample

```
portal.close()
```

Behavior: - Clears cookies and cache for the portal URL - Dismisses the portal with slide animation - Hides status bar during dismissal - Triggers `closeEvent` to event listeners

5.4 addEventListener

Registers an event listener for the specified event name.

Signature

```
func addEventListener(eventName: String, listener: @escaping EventListener)
```

Type Definition

```
public typealias EventListener = (AnyObject) -> [String: Any]?
```

Parameters

Parameter	Type	Description
eventName	String	Name of the event to listen for
listener	EventListener	Callback function that receives event data and optionally returns a response

Sample

```
portal.addEventListener(eventName: "eventsCallbacks") { data in
    print("Listener received event data: \(data)")

    if let eventData = data as? [String: Any],
       let name = eventData["name"] as? String {

        if name == "close" {
            print("Handling close event")
            portal.close()
        } else if name == "token-expired" {
            print("Handling token expiry")
            // Return new token to SDK
            return ["accessToken": "new-access-token"]
        }
    }
    return nil
}
```

Event Retention: Events dispatched before listeners are registered will be queued and replayed when the listener is added. This ensures no events are lost during initialization.

Common Event Names: - `"eventsCallbacks"` - Main event channel for web-to-native communication - `"closeEvent"` - Fired when portal is closed - `"urlChangeEvent"` - Fired when URL changes - `"confirmBtnClicked"` - Fired when share disclaimer is confirmed

5.5 notifyListener

Sends data to the web layer via the event system.

Signature

```
func notifyListener(eventName: String, data: [String: Any]) -> Void
```

Parameters

Parameter	Type	Description
eventName	String	Event name (currently only "token-updated" is supported)
data	[String: Any]	Data to send to web layer

Sample

```
let newConfiguration: [String: Any] = [  
    "accessToken": "new-access-token"  
]  
portal.notifyListener(eventName: "token-updated", data: newConfiguration)
```

Behavior: - For "token-updated" event: - Updates the `accessToken` cookie (HttpOnly) - Dispatches `cap-events-response` event to web with the data - Web can listen for this event to receive the new token

Appendix: Migration from v0.4 to v0.5

If you're upgrading from version 0.4, here are the key changes:

New Features

1. **Geolocation Support** - Web content can use `navigator.geolocation` to access device location (iOS 15+)

Info.plist Requirements

Add the following key if your web content uses geolocation:

```
<key>NSLocationWhenInUseUsageDescription</key>  
<string>We need location access to [explain your use case]</string>
```

Breaking Changes

None - v0.5 is fully backward compatible with v0.4

Recommended Updates

1. Add `NSLocationWhenInUseUsageDescription` to Info.plist if your web content uses geolocation

2. No native code changes required — the SDK handles geolocation permission automatically

Support

For questions or issues with the HASPortal SDK, please contact: - **Email:** [support contact] -
Documentation: [documentation URL]

End of Document